SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	**Assessment:**	Industry Problem solving task
Weightage:	30%	**Total Marks:**	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	Moniga V	**Reg. No.:**	192521184
Section / Batch:	2025 - 2029	**Date:**	19/08/2026

Instructions to Students

- Identify the relevant concepts of cache hierarchy, SRAM, DRAM, MRAM, NAND Flash, RRAM, memory latency, bandwidth, and virtual memory paging.
- Analyze the memory-access requirements of smartphones, cloud servers, autonomous vehicles, edge AI devices, and gaming consoles, considering latency, bandwidth, capacity, and power consumption.
- Apply suitable memory-hierarchy techniques such as cache optimization, appropriate memory technology selection, data locality, paging optimization, and hierarchical memory organization.
- Compute performance measures such as memory access time, latency, bandwidth, throughput, speedup, hit/miss rates, and resource or power utilization where applicable.
- Interpret the results by evaluating performance improvements, memory bottlenecks, technology trade-offs, and the suitability of the proposed memory hierarchy for each application.

QUESTIONS

1. A mobile device manufacturer observes noticeable lag when users switch rapidly between multiple applications. Analyze how L1, L2, and L3 caches and DRAM participate in storing and retrieving frequently accessed application data. Based on latency, capacity, hit rate, and bandwidth requirements, propose a suitable hierarchical memory organization that can reduce applicationswitching delays and improve overall system throughput.
2. A cloud service provider experiences increased latency when processing database queries from a large number of concurrent users. Analyze how cache hierarchy and virtual memory paging influence memory access performance, and compare their roles in reducing processor-memory latency. Recommend suitable memory technologies and memory-management improvements that can minimize page faults, reduce access delays, and improve database throughput.
3. An autonomous vehicle continuously receives large volumes of sensor data from cameras, LiDAR, radar, and other real-time sensing systems. Evaluate the characteristics of SRAM, DRAM, and MRAM in terms of access latency, bandwidth, capacity, volatility, and power consumption. Based on the real-time processing requirements, design an appropriate memory hierarchy that can provide high data-transfer bandwidth while minimizing access latency and supporting reliable sensor-data processing.
4. An edge-based AI device must load machine-learning models quickly while operating under strict power and storage constraints. Analyze the characteristics of NAND Flash, DRAM, and RRAM with respect to access speed, storage capacity, persistence, power consumption, and suitability for AI workloads. Recommend a hierarchical memory organization that enables fast model loading, efficient intermediate-data processing, reduced energy consumption, and improved overall inference performance.
5. A high-performance gaming console experiences frame drops and noticeable delays when loading large game environments containing high-resolution textures, audio, and other assets. Analyze the interaction between L3 cache and DRAM during the retrieval and processing of frequently accessed game data, considering their latency, capacity,

and bandwidth. Propose suitable memoryhierarchy enhancements that can reduce data-access bottlenecks, increase memory throughput, and provide smoother frame rendering during large scene transitions.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Q. No	Understanding of Industry Problem (20)	Application of Engineering Concepts (25)	Solution Design (25)	Use of Tools (15)	Analysis (10)	Professionalism(5)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/5	/ 100

Course Coordinator **Faculty Incharge**

Answer 1: Reducing Application-Switching Lag on Mobile Devices

1. Understanding of the Industry Problem

Frequent app-switching lag on smartphones is caused by the processor repeatedly reloading application state (code, UI resources, and working data) from slower memory because it is no longer present in fast on-chip storage. The core constraint is that mobile SoCs have very limited silicon area and power budget for cache, so capacity, latency, and energy must all be balanced simultaneously while still meeting real-time UI responsiveness expectations (target frame times under ~16 ms for 60 Hz displays).

2. Role of L1, L2, L3 Cache and DRAM

- L1 Cache (32–64 KB, split I/D): closest to the CPU core, access latency ~1–4 cycles. Holds the immediate instruction stream and hot registers/variables of the foreground app's active thread. Highest hit rate needed for per-instruction fetch/decode.
- L2 Cache (256 KB–1 MB per core cluster): access latency ~10–20 cycles. Buffers recently used code/data that no longer fits in L1, including UI-rendering routines and frequently touched heap objects across the app's threads.
- L3 / Shared System Cache (2–8 MB, shared across CPU/GPU clusters): access latency ~30–60 cycles. Retains data shared between the app that is losing foreground focus and the one gaining it (e.g., common OS services, shared libraries, compositor buffers), reducing the penalty of a context switch.
- DRAM (LPDDR4X/LPDDR5, several GB): access latency ~100–300 cycles (tens of ns), bandwidth 20–50+ GB/s. Stores each app's full working set (heap, stack, loaded resources). When an app is switched back in and its data has been evicted from all cache levels, the OS must re-fetch from DRAM or, in the worst case, from NAND storage (swap/compressed memory), which is the primary source of visible lag.

3. Proposed Hierarchical Memory Organization

Increase L2 capacity per core cluster (e.g., 512 KB → 1 MB) and enlarge the shared L3/SLC to around 6–8 MB so that the working sets of 2–3 recently used apps can coexist without full eviction, directly targeting fast app-resume.

Use way-partitioning or priority hints in the shared L3 so the foreground app is guaranteed cache ways while background apps retain a smaller reserved portion — this avoids one app fully evicting another's cached state during a switch.

Employ a victim-cache / exclusive L2–L3 policy (data present in L2 is not duplicated in L3) to maximize effective on-chip capacity without added silicon.

Use LPDDR5/5X DRAM with higher burst bandwidth and finer-grained low-power states so that when a cache miss does occur, the DRAM fetch is fast and the memory controller can still power-gate idle banks for battery life.

At the OS level, prioritize keeping recently backgrounded apps' memory pages resident (rather than immediately reclaiming them) using an LRU-based app-freeze policy, so DRAM — not flash-backed swap — is the fallback on switch-back.

4. Performance Computation (Illustrative)

Average Memory Access Time (AMAT) for the current 2-level design: AMAT = L1_hit_time + L1_miss_rate × (L2_hit_time + L2_miss_rate × DRAM_time). Using L1 = 1.5 ns, L1 miss rate = 8%, L2 = 6 ns, L2 miss rate = 25%, DRAM = 100 ns: AMAT = 1.5 + 0.08 × (6 + 0.25 × 100) = 1.5 + 0.08 × 31 = 1.5 + 2.48 = 3.98 ns.

With the proposed 3-level hierarchy (added L3, L2 miss rate reduced to 25% of which only 15% now misses to DRAM, the rest served by L3 at 15 ns): AMAT = 1.5 + 0.08 × (6 + 0.25 × (15 + 0.15 × 100)) = 1.5 + 0.08 × (6 + 0.25 × 30) = 1.5 + 0.08 × 13.5 = 1.5 + 1.08 = 2.58 ns.

Speedup = 3.98 / 2.58 ≈ 1.54×, i.e., roughly a 35% reduction in average access time, which translates directly into fewer dropped frames and shorter perceptible delay during app switches.

5. Interpretation / Suitability

The bottleneck for app-switching lag is predominantly capacity-driven cache eviction, not raw DRAM speed. Adding a shared, way-partitioned L3 gives the greatest marginal improvement per die-area/power cost, while DRAM upgrades (LPDDR5X) provide a secondary but still meaningful gain for the residual miss traffic. This hierarchy is well suited to mobile constraints because it improves hit rate (and therefore energy per access) rather than simply increasing clock speed, which is important given the strict thermal and battery envelope of a smartphone.

Answer 2: Reducing Query Latency for Cloud Database Servers

1. Understanding of the Industry Problem

Cloud database servers must serve many concurrent users' queries with unpredictable, often random, memory-access patterns across a working set (indexes, buffer pool pages, query results) far larger than any on-chip cache. Two related bottlenecks appear: (a) low cache hit rates due to contention between many concurrent query threads thrashing shared cache lines, and (b) virtual-memory page faults when the active working set exceeds physical DRAM, forcing slow disk/SSD-backed page-ins.

2. Cache Hierarchy vs Virtual Memory Paging

Cache hierarchy (L1/L2/L3) reduces processor-to-memory latency for hot, frequently reused data such as B-tree index nodes and lock/metadata structures; it operates at cache-line granularity (64 B) and nanosecond latency, but shared L3 across many cores is easily thrashed when hundreds of concurrent connections each touch different data.

Virtual memory paging manages the much larger gap between DRAM capacity and total dataset size, operating at page granularity (4 KB–2 MB) with the TLB/page table translating addresses. A page fault triggers disk or network-storage I/O, costing on the order of 100,000+ ns — several orders of magnitude slower than even a DRAM access — making it the dominant source of tail latency under concurrency.

In short, cache hierarchy optimizes for temporal/spatial locality at nanosecond scale, while paging determines whether the needed data is resident in DRAM at all; a well-tuned cache cannot compensate for a high page-fault rate, so both must be addressed together.

3. Recommended Memory Technologies and Management Improvements

Increase DRAM capacity (favoring DDR5/HBM-adjacent server memory with higher bandwidth) so the active buffer pool and hot indexes remain resident, cutting page-fault frequency at the source.

Use large/huge pages (2 MB or 1 GB) for the database buffer pool to reduce TLB miss rate and page-table walk overhead, which is a major contributor to per-query latency at high concurrency.

Deploy NVMe SSDs (or Optane/persistent-memory as an intermediate tier) as the swap/backing tier instead of traditional HDDs, reducing page-fault service time by roughly two orders of magnitude compared to spinning disk.

Apply cache-partitioning technologies (e.g., Intel CAT-style shared-cache allocation) so noisy-neighbor query threads cannot evict another tenant's hot index pages from L3, improving effective hit rate under concurrency.

Use working-set-aware page replacement (e.g., CLOCK/ARC variants) in the OS or DBMS buffer-pool manager rather than plain LRU, since database access patterns are often scan-heavy and can defeat naive LRU.

4. Performance Computation (Illustrative)

Effective Access Time (EAT) with paging: EAT = (1 − page_fault_rate) × memory_access_time + page_fault_rate × page_fault_service_time. With memory_access_time = 100 ns, page_fault_service_time = 150,000 ns (NVMe-backed), and fault rate reduced from 0.01% to 0.002% via larger DRAM/huge pages: EAT_before = 0.9999×100 + 0.0001×150000 = 99.99 + 15 = 114.99 ns; EAT_after = 0.99998×100 + 0.00002×150000 = 99.998 + 3 = 102.99 ns — about a 10% reduction, but critically this reduces worst-case tail latency far more than average, which is what matters for concurrent user experience.

Throughput improvement can be approximated as inversely proportional to EAT: Throughput_ratio ≈ 114.99 / 102.99 ≈ 1.12×, i.e., roughly a 12% increase in queries served per second for the same hardware.

5. Interpretation / Suitability

For cloud database workloads, the highest-leverage fix is reducing the page-fault rate (via larger DRAM and huge pages) because each fault costs orders of magnitude more than any cache-level optimization. Cache partitioning is the secondary, complementary fix that protects hit rate under multi-tenant contention. Together these directly target the reported symptom — latency that grows with the number of concurrent users — because both mechanisms specifically mitigate contention-driven misses and faults rather than just raw clock speed.

Answer 3: Memory Hierarchy for Autonomous Vehicle Sensor Processing

1. Understanding of the Industry Problem

An autonomous vehicle's perception stack ingests continuous, high-bandwidth streams from cameras, LiDAR, and radar and must process them within strict real-time deadlines (typically under 100 ms end-to-end for safety-critical decisions). The memory system must simultaneously deliver very high sustained bandwidth for streaming sensor data and very low, deterministic latency for the safety-critical control loop, while managing power and reliability within an automotive-grade thermal envelope.

2. Evaluation of SRAM, DRAM, and MRAM

- SRAM: sub-nanosecond to low-nanosecond latency, very high bandwidth per bit, but low density and high cost/area, and it is volatile (loses data on power loss). Best suited for on-chip buffers, register files, and small deterministic scratchpads used directly by the perception/control pipeline.
- DRAM (e.g., LPDDR5/GDDR6 in automotive SoCs): moderate latency (tens of ns) but very high aggregate bandwidth (tens to hundreds of GB/s) and much higher density than SRAM at lower cost per bit; volatile. Well suited as the main frame-buffer/working-memory tier for holding raw and intermediate sensor-fusion data (e.g., point clouds, image tensors).
- MRAM: near-SRAM speed with non-volatility, moderate density (better than SRAM, generally less than DRAM), and much lower standby power since it retains state without refresh. Its non-volatility is valuable for safety-critical state (e.g., last-known vehicle state, fault logs) that must survive power glitches — a key automotive reliability requirement.

3. Proposed Memory Hierarchy

Tier 1 — On-chip SRAM scratchpad/cache close to each sensor-processing core (camera ISP, LiDAR point-cloud engine, radar signal processor) for deterministic, low-latency buffering of the immediate incoming frame/scan, ensuring the real-time deadline is met independent of shared-bus contention.

Tier 2 — High-bandwidth DRAM (LPDDR5X or GDDR6-class) as the shared working memory for sensor-fusion, holding multiple recent frames/point-clouds and intermediate neural-network activations, sized for sustained multi-gigabyte-per-second streaming from all sensors concurrently.

Tier 3 — MRAM as a small non-volatile buffer for critical state (last valid perception output, safety flags, black-box style event logging) so that on a power interruption the vehicle retains the information needed for a safe-stop decision without waiting on flash write latency.

Use dedicated, isolated memory channels/ports per sensor class (camera vs LiDAR vs radar) into the DRAM controller to prevent bandwidth contention from one sensor stream starving another, which is critical since all three feed the same real-time fusion algorithm.

4. Performance Computation (Illustrative)

Required sustained bandwidth: 8 cameras × 1920×1080×3 bytes × 30 fps ≈ 8 × 186 MB/s ≈ 1.5 GB/s for cameras alone; adding LiDAR (~100 MB/s) and radar (~10–50 MB/s) gives a combined requirement of roughly 1.7 GB/s sustained, well within a single LPDDR5X channel (~34 GB/s) or GDDR6 (~500+ GB/s aggregate), confirming DRAM tier sizing is adequate with margin for burst fusion workloads.

Latency budget check: if the control loop deadline is 10 ms and SRAM buffering plus DRAM fusion access consumes ~1–2 ms (SRAM ~ns-scale, DRAM burst transfer for a full frame ~hundreds of µs), the remaining ~8 ms budget is available for compute, showing the proposed hierarchy does not itself become the bottleneck relative to the real-time deadline.

5. Interpretation / Suitability

The SRAM–DRAM–MRAM combination balances the three competing needs of this application: deterministic low latency (SRAM), high sustained streaming bandwidth (DRAM), and power-safe non-volatile retention of safety-critical state (MRAM). This hierarchy is well suited because it isolates the deadline-critical path (SRAM) from the bulk-bandwidth path (DRAM) while adding a resilience layer (MRAM) that pure SRAM/DRAM designs lack — directly addressing the reliability requirement inherent to autonomous driving.

Answer 4: Memory Hierarchy for Edge AI Model Loading and Inference

1. Understanding of the Industry Problem

Edge AI devices must load and run machine-learning models under tight power and storage budgets, unlike cloud accelerators with abundant DRAM/HBM. The key tension is between persistence and capacity (needed to store the model when powered off) and speed and energy-efficiency (needed for fast loading and low-latency inference), all constrained by a small battery or thermal envelope.

2. Evaluation of NAND Flash, DRAM, and RRAM

- NAND Flash: high density and low cost per bit, non-volatile, but relatively slow access (tens of µs read latency) and limited write endurance; well suited for persistently storing the trained model weights when the device is powered off.
- DRAM: fast (tens of ns) and high bandwidth, but volatile and requires continuous refresh power; suited as the working memory where the model is loaded for active inference and where intermediate activations/tensors are computed.
- RRAM: non-volatile like NAND Flash but with much lower latency (comparable to DRAM in many implementations) and lower write energy; particularly attractive for in-memory/near-memory computing of neural-network weights and for constrained devices that need instant-on inference without a full DRAM reload.

3. Proposed Hierarchical Memory Organization

Store the trained model persistently in NAND Flash for cost-effective bulk storage of the full model and any spare/versioned models.

Use RRAM as a fast non-volatile cache for the most frequently used model (or its most latency-critical layers), enabling near-instant model availability on wake without the full NAND read/decompress penalty — directly addressing 'fast model loading'.

Use DRAM as the active execution memory for intermediate activations and the currently running layer's working tensors, since these change every inference cycle and benefit from DRAM's higher write endurance and bandwidth compared to RRAM/NAND.

Apply model compression/quantization (e.g., INT8) before storage to shrink the NAND footprint and reduce the volume of data that must move from NAND/RRAM into DRAM at load time, directly cutting both load latency and energy.

Use power-gating on DRAM banks not currently in use between inference bursts, relying on RRAM's non-volatility to preserve the hot model weights instead of keeping all of DRAM continuously refreshed.

4. Performance Computation (Illustrative)

Model load time from NAND (cold start): for a 50 MB quantized model at NAND sequential read ≈ 200 MB/s effective, load time ≈ 50/200 = 0.25 s.

Model load time from RRAM cache (warm start): at RRAM read bandwidth ≈ 1 GB/s (typical for emerging RRAM arrays) and comparable size, load time ≈ 50/1000 = 0.05 s — a 5× reduction in load latency for the common case where the model is already cached in RRAM.

Energy comparison (illustrative): if NAND read energy ≈ 10 nJ/bit and RRAM read energy ≈ 1–2 nJ/bit, moving the repeated 'hot model' load path to RRAM can cut per-load energy by roughly 5–10×, which is significant for a battery-powered edge device performing many inference sessions per day.

5. Interpretation / Suitability

The NAND–RRAM–DRAM hierarchy is well suited to edge AI because it separates the rarely-changing bulk storage role (NAND, cheapest per bit), the latency-critical hot-model role (RRAM, fast and non-volatile), and the high-churn compute-working role (DRAM, fastest and highest endurance). This directly improves inference performance by minimizing both the time and energy spent moving model data before each inference burst, which is the dominant overhead in edge deployments with intermittent usage patterns.

Answer 5: Reducing Frame Drops in a Gaming Console via L3 Cache and DRAM Optimization

1. Understanding of the Industry Problem

Frame drops during large scene transitions occur when the GPU/CPU pipeline cannot retrieve high-resolution texture, geometry, and audio assets fast enough to meet the frame budget (e.g., 16.7 ms for 60 fps, 6.9 ms for 144 fps). The bottleneck is the interaction between the shared L3 cache, which holds recently used asset metadata and small hot textures, and DRAM/GDDR memory, which must stream in the much larger bulk of new assets as the scene changes — a classic capacity-vs-bandwidth trade-off.

2. Interaction Between L3 Cache and DRAM

L3 cache (often 8–32 MB shared across CPU/GPU in modern console SoCs) provides low-latency (tens of cycles) access to frequently reused small data structures — draw-call metadata, shader constants, and recently touched texture tiles — improving hit rate for assets that persist across frames within the same scene.

DRAM/GDDR (10–16+ GB, hundreds of GB/s bandwidth) holds the full asset set for the current and upcoming scenes; during a scene transition, a large volume of new textures and geometry must be streamed from storage into DRAM and then touched by the GPU, causing a burst of L3 misses (cold-cache effect) exactly when the frame budget is tightest.

The frame-drop symptom arises because L3's limited capacity cannot hold the entirely new working set introduced by a scene transition, forcing a spike in DRAM traffic that, combined with normal rendering bandwidth demand, can exceed available bandwidth or introduce latency stalls in the GPU pipeline.

3. Proposed Memory-Hierarchy Enhancements

Increase effective L3 capacity or add a victim/system-level cache dedicated to texture streaming, so that assets prefetched just ahead of a scene transition remain resident rather than being evicted by concurrent CPU traffic.

Implement predictive/asynchronous asset streaming (prefetching textures and geometry into DRAM slightly ahead of when the renderer needs them, based on player position/camera direction) to convert transition-time bandwidth spikes into steadier, pre-scheduled DRAM traffic.

Use texture compression (e.g., BCn/ASTC formats) so the same visual fidelity requires less DRAM bandwidth and L3 capacity per asset, directly reducing the data volume that must move during a transition.

Partition GDDR/DRAM bandwidth between CPU and GPU access streams (or use priority-based memory-controller QoS) so that bulk asset streaming does not starve latency-sensitive rendering/audio accesses during the transition window.

Use high-bandwidth GDDR6/6X (or an on-package high-bandwidth memory tier) sized so peak bandwidth during a scene transition still leaves headroom above the sustained bandwidth needed for steady-state rendering.

4. Performance Computation (Illustrative)

Bandwidth requirement during transition: loading 500 MB of new texture/geometry data within a 2-second transition window requires 500 MB / 2 s = 250 MB/s minimum, but if it must be interleaved with steady-state rendering traffic of, say, 300 GB/s peak GDDR6 bandwidth, the transition load is a small fraction (~0.08%) of peak bandwidth — showing that with proper scheduling/prefetch, bandwidth itself is rarely the hard limit; miss-driven latency stalls are the real issue.

AMAT improvement from added L3 capacity: with L2/L3 miss rate for texture accesses reduced from 40% to 20% due to a larger cache (L3 access = 40 cycles, DRAM access = 250 cycles): AMAT_before

= 0.40 × 250 + 0.60 × 40 = 100 + 24 = 124 cycles; AMAT_after = 0.20 × 250 + 0.80 × 40 = 50 + 32 = 82 cycles — a speedup of 124/82 ≈ 1.51×, meaning texture-access stalls during the transition are reduced by roughly a third, directly translating to fewer dropped frames.

5. Interpretation / Suitability

Because raw DRAM bandwidth is generally sufficient on modern consoles, the actual cause of frame drops during transitions is predominantly cache-miss latency and unscheduled burst traffic rather than a hard bandwidth ceiling. Enhancements that increase effective cache capacity, enable predictive prefetching, and add memory-controller QoS therefore give the largest practical improvement, making this hierarchy well suited to eliminating transition-time frame drops without requiring a full memory subsystem redesign.